

QUINDIOFLIX- Plataforma de Streaming de Contenido Multimedia
Documento de Sustentación Entrega 3
Versión 1.0

Preparado por:
Jhon Sebastian Alzate Andica
Luis Manuel Osorio Medina
Ismenia Marcela Guevara Ortiz

Universidad del Quindío
Facultad de Ingeniería de Sistemas y Computación
Armenia - Quindío
2026

Transacciones

Las tres transacciones fueron diseñadas para cubrir los ciclos de vida más críticos de un usuario dentro del sistema QuindioFlix, garantizando integridad de datos en operaciones que involucren múltiples tablas y pasos encadenados, el código sql es anexado a los archivos de esta entrega con el nombre: Transacciones.sql

Transacción 1: Registro completo de usuario

Gestiona el proceso de incorporación de un nuevo suscriptor al sistema de forma atómica, asegurando que el usuario, su perfil principal y su primer pago queden registrados todos juntos o ninguno. Si cualquier paso falla, el rollback al savepoint deja la base de datos como si el registro nunca hubiera ocurrido.

```
=== TRANSACCION 1: REGISTRO COMPLETO DE USUARIO ===  
Usuario registrado correctamente.  
Usuario creado con id 31  
Perfil creado con id 62  
Pago inicial registrado con id 106  
TRANSACCION 1 CONFIRMADA  
  
Procedimiento PL/SQL terminado correctamente.
```

Escenario de concurrencia: Dos sesiones intentan registrar simultáneamente un usuario con el mismo correo electrónico.

La Sesión A ejecuta sp_registrar_usuario e inserta el usuario pero aún no hace COMMIT.

La Sesión B intenta registrar el mismo email y queda bloqueada por el lock de fila que mantiene A.

Si A hace COMMIT, B falla con violación de restricción UNIQUE sobre el email.

Si A hace ROLLBACK, B puede continuar y completar su registro.

Problema potencial: Sin el SAVEPOINT sp_registro_inicio, un fallo en el paso 3 (perfil) o paso 4 (pago) dejaría un usuario huérfano sin perfil ni pago registrado. El rollback al savepoint garantiza que los tres pasos sean atómicos.

Transacción 2: Renovación mensual masiva

Procesa el cobro periódico de todos los usuarios con cuenta activa, calculando el monto individual de cada uno y actualizando su fecha de vencimiento. Usa un savepoint por usuario para que un fallo individual no afecte las renovaciones ya procesadas, garantizando que el mayor número posible de usuarios sea renovado exitosamente en cada ejecución.

```
=== TRANSACCION 2: RENOVACION MENSUAL ===
Renovacion OK usuario 2 monto 34900
Renovacion OK usuario 3 monto 24900
Renovacion OK usuario 4 monto 14900
Renovacion OK usuario 5 monto 24900
Renovacion OK usuario 6 monto 24900
Renovacion OK usuario 7 monto 22410
Renovacion OK usuario 8 monto 13410
Renovacion OK usuario 10 monto 22410
Renovacion OK usuario 11 monto 13410
Renovacion OK usuario 12 monto 14900
Renovacion OK usuario 13 monto 31410
Renovacion OK usuario 14 monto 13410
Renovacion OK usuario 15 monto 34900
Renovacion OK usuario 16 monto 13410
Renovacion OK usuario 18 monto 24900
Renovacion OK usuario 19 monto 13410
Renovacion OK usuario 20 monto 13410
Renovacion OK usuario 21 monto 34900
Renovacion OK usuario 22 monto 13410
Renovacion OK usuario 23 monto 22410
Renovacion OK usuario 24 monto 24900
Renovacion OK usuario 26 monto 31410
Renovacion OK usuario 27 monto 14900
Renovacion OK usuario 29 monto 13410
Renovacion OK usuario 30 monto 24900
Renovacion OK usuario 31 monto 14900
Transaccion 2 CONFIRMADA (usuarios procesados)

Procedimiento PL/SQL terminado correctamente.
```

Escenario de concurrencia: El proceso de renovación masiva corre al mismo tiempo que un usuario actualiza su plan o cancela su cuenta.

La Sesión A (renovación) recorre el cursor de usuarios activos y procesa al usuario 5.

La Sesión B (usuario o admin) intenta cambiar el estado o plan del mismo usuario 5 en paralelo.

A tiene un lock sobre la fila del usuario 5 durante el UPDATE, por lo que B queda en espera hasta que A libere ese lock con el COMMIT final o el ROLLBACK TO sp_usuario.

Problema potencial: Si fn_calcular_monto devuelve NULL para algún usuario, el ROLLBACK TO sp_usuario revierte solo ese registro y el proceso continúa con los demás, evitando que un error individual anule toda la renovación masiva. Sin los savepoints por usuario, un solo fallo tumbaría todos los pagos ya insertados.

Transacción 3: Eliminación completa de cuenta

Elimina de forma total e irreversible todos los datos asociados a un usuario: pagos, reportes y, por cascada, perfiles, reproducciones, favoritos y calificaciones. Opera bajo un esquema de todo o nada, de modo que si algún paso falla, se hace rollback y el usuario permanece intacto en el sistema, evitando registros huérfanos o eliminaciones parciales.

```
=== TRANSACCION 3: ELIMINACION DE CUENTA ===  
Transaccion 3 CONFIRMADA. Usuario 31 eliminado.  
  
Procedimiento PL/SQL terminado correctamente.
```

Escenario de concurrencia: Una sesión elimina al usuario mientras otra sesión está consultando o modificando datos relacionados a ese mismo usuario.

La Sesión A inicia la eliminación: borra pagos y reportes del usuario 31, luego el DELETE FROM usuario.

La Sesión B intenta simultáneamente registrar una reproducción o calificación para el perfil de ese mismo usuario.

B queda bloqueada porque A tiene un lock exclusivo sobre la fila del usuario 31.

Una vez A hace COMMIT, el ON DELETE CASCADE ya habrá eliminado perfiles, reproducciones, favoritos y calificaciones en cadena, por lo que B fallará con un error de integridad referencial.

Problema potencial: Si A falla a mitad del proceso (por ejemplo al borrar reporte_contenido), el ROLLBACK TO sp_eliminar restaura todos los datos del usuario intactos, garantizando que nunca quede una eliminación parcial en el sistema.

Índices

En el proyecto se contempló desde el diseño del esquema la creación de índices estratégicos para optimizar las consultas más críticas del sistema. A continuación se listan los índices seleccionados y analizados mediante EXPLAIN PLAN FOR:

- IX_REPRODUCCION_PERFIL
- IX_USUARIO_CIUDAD
- IX_PAGO_ESTADO
- IX_PAGO_FECHA
- IX_CONTENIDO_TIPO
- IX_USUARIO_PLAN
- IX_REPRODUCCION_CONTENIDO

La selección de estos índices responde a los patrones de consulta más frecuentes e importantes del sistema: historial de reproducciones, análisis de usuarios por ciudad, reportes de ingresos y estadísticas de consumo de contenido. Su impacto fue evaluado comparando los planes de ejecución con y sin índices activos, evidenciando mejoras de costo de hasta un 60% en algunas consultas.

Les cambiamos el estado a UNUSABLE antes de ejecutar el explain plan for sin índices en las consultas

```
ALTER INDEX ix_reproduccion_perfil      UNUSABLE;  
ALTER INDEX ix_usuario_ciudad          UNUSABLE;  
ALTER INDEX ix_pago_estado             UNUSABLE;  
ALTER INDEX ix_pago_fecha              UNUSABLE;  
ALTER INDEX ix_contenido_tipo          UNUSABLE;  
ALTER INDEX ix_usuario_plan            UNUSABLE;  
ALTER INDEX ix_reproduccion_contenido  UNUSABLE;  
/
```

El estado de los índices vuelve a cambiar con rebuild antes de ejecutar el explain plan for con índices en las consultas

```
ALTER INDEX ix_reproduccion_perfil      REBUILD;  
ALTER INDEX ix_usuario_ciudad          REBUILD;  
ALTER INDEX ix_pago_estado             REBUILD;  
ALTER INDEX ix_pago_fecha              REBUILD;  
ALTER INDEX ix_contenido_tipo          REBUILD;  
ALTER INDEX ix_usuario_plan            REBUILD;  
ALTER INDEX ix_reproduccion_contenido  REBUILD;  
/
```

Consultas para analizar el explain plan for antes y después:

Consulta 1 Historial de reproducciones por perfil y rango de fechas

Obtiene todas las reproducciones del perfil con id_perfil = 1 durante el año 2025, mostrando detalles como dispositivo, porcentaje de avance y tiempos de inicio/fin, ordenadas cronológicamente.

```
-- Limpiar plan
DELETE FROM plan_table;

-- 1) Historial de reproducciones por perfil y rango de fechas
EXPLAIN PLAN FOR
SELECT
    r.id_reproduccion,
    r.id_perfil,
    r.id_contenido,
    r.fecha_hora_inicio,
    r.fecha_hora_fin,
    r.dispositivo,
    r.porcentaje_avance
FROM   reproduccion r
WHERE  r.id_perfil = 1
AND    r.fecha_hora_inicio BETWEEN
        TO_TIMESTAMP('2025-01-01 00:00:00', 'YYYY-MM-DD HH24:MI:SS')
        AND TO_TIMESTAMP('2025-12-31 23:59:59', 'YYYY-MM-DD HH24:MI:SS')
ORDER BY r.fecha_hora_inicio;

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
/
```

Antes

PLAN_TABLE_OUTPUT

Plan hash value: 4184488331

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		2	86	4 (25)	00:00:01
1	SORT ORDER BY		2	86	4 (25)	00:00:01
* 2	TABLE ACCESS FULL	REPRODUCCION	2	86	3 (0)	00:00:01

Predicate Information (identified by operation id):

```
2 - filter("R"."ID_PERFIL"=1 AND "R"."FECHA_HORA_INICIO"<=TIMESTAMP'
      2025-12-31 23:59:59.000000000' AND "R"."FECHA_HORA_INICIO">=TIMESTAMP'
      2025-01-01 00:00:00.000000000')
```

Después

PLAN_TABLE_OUTPUT

Plan hash value: 609622800

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		2	86	3 (34)	00:00:01
1	SORT ORDER BY		2	86	3 (34)	00:00:01
* 2	TABLE ACCESS BY INDEX ROWID BATCHED	REPRODUCCION	2	86	2 (0)	00:00:01
* 3	INDEX RANGE SCAN	IX_REPRODUCCION_PERFIL	4		1 (0)	00:00:01

Predicate Information (identified by operation id):

```
2 - filter("R"."FECHA_HORA_INICIO"<=TIMESTAMP' 2025-12-31 23:59:59.000000000' AND
      "R"."FECHA_HORA_INICIO">=TIMESTAMP' 2025-01-01 00:00:00.000000000')
3 - access("R"."ID_PERFIL"=1)
```

Consulta 2 Usuarios por ciudad

Agrupar todos los usuarios según su ciudad de residencia y cuenta cuántos hay en cada una, ordenando de mayor a menor.

```
-- 2) Usuarios por ciudad con conteo, usando ciudad_residencia
DELETE FROM plan_table;

EXPLAIN PLAN FOR
SELECT
    u.ciudad_residencia,
    COUNT(*) AS total_usuarios
FROM    usuario u
GROUP BY u.ciudad_residencia
ORDER BY total_usuarios DESC;

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
/
```

Antes

PLAN_TABLE_OUTPUT

Plan hash value: 1514796836

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		5	45	5 (40)	00:00:01
1	SORT ORDER BY		5	45	5 (40)	00:00:01
2	HASH GROUP BY		5	45	5 (40)	00:00:01
3	TABLE ACCESS FULL	USUARIO	31	279	3 (0)	00:00:01

Después

PLAN_TABLE_OUTPUT

Plan hash value: 515400797

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		5	45	2 (50)	00:00:01
1	SORT ORDER BY		5	45	2 (50)	00:00:01
2	SORT GROUP BY NOSORT		5	45	2 (50)	00:00:01
3	INDEX FULL SCAN	IX_USUARIO_CIUADAD	31	279	1 (0)	00:00:01

Consulta 3 Ingresos por ciudad y plan en un año

Suma los montos pagados exitosamente durante 2025, agrupados por ciudad y tipo de plan de suscripción, para ver qué combinación ciudad/plan generó más ingresos.

```
-- 3) Ingresos por ciudad y plan en un año (usa estado_pago y fecha_pago)
DELETE FROM plan_table;

EXPLAIN PLAN FOR
SELECT
    u.ciudad_residencia,
    pl.nombre          AS plan,
    SUM(pg.monto_pagado) AS ingresos_totales
FROM   pago_suscripcion pg
JOIN   usuario          u ON u.id_usuario = pg.id_usuario
JOIN   plan_suscripcion pl ON pl.id_plan   = u.id_plan
WHERE  pg.estado_pago = 'EXITOSO'
AND    pg.fecha_pago BETWEEN
        TO_TIMESTAMP('2025-01-01 00:00:00', 'YYYY-MM-DD HH24:MI:SS')
        AND TO_TIMESTAMP('2025-12-31 23:59:59', 'YYYY-MM-DD HH24:MI:SS')
GROUP BY u.ciudad_residencia, pl.nombre
ORDER BY u.ciudad_residencia, plan;

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
/
```

Antes

PLAN_TABLE_OUTPUT

Plan hash value: 1461881374

Id	Operation	Name	Rows	Bytes	Cost	(%CPU)	Time
0	SELECT STATEMENT		11	583	10	(20)	00:00:01
1	SORT ORDER BY		11	583	10	(20)	00:00:01
2	HASH GROUP BY		11	583	10	(20)	00:00:01
* 3	HASH JOIN		102	5406	8	(0)	00:00:01
* 4	HASH JOIN		31	806	5	(0)	00:00:01
5	VIEW	index\$_join\$_004	3	33	2	(0)	00:00:01
* 6	HASH JOIN						
7	INDEX FAST FULL SCAN	PK_PLAN_SUSCRIPCION	3	33	1	(0)	00:00:01
8	INDEX FAST FULL SCAN	UQ_PLAN_NOMBRE	3	33	1	(0)	00:00:01
9	TABLE ACCESS FULL	USUARIO	31	465	3	(0)	00:00:01
* 10	TABLE ACCESS FULL	PAGO_SUSCRIPCION	102	2754	3	(0)	00:00:01

Predicate Information (identified by operation id):

```

3 - access("U"."ID_USUARIO"="PG"."ID_USUARIO")
4 - access("PL"."ID_PLAN"="U"."ID_PLAN")
6 - access(ROWID=ROWID)
10 - filter("PG"."FECHA_PAGO"<=TIMESTAMP' 2025-12-31 23:59:59.000000000' AND
          "PG"."ESTADO_PAGO"='EXITOSO' AND "PG"."FECHA_PAGO">=TIMESTAMP' 2025-01-01
          00:00:00.000000000')
```

Note

- this is an adaptive plan

Después

PLAN_TABLE_OUTPUT

Plan hash value: 1553672496

Id	Operation	Name	Rows	Bytes	Cost	(%CPU)	Time
0	SELECT STATEMENT		11	583	10	(20)	00:00:01
1	SORT ORDER BY		11	583	10	(20)	00:00:01
2	HASH GROUP BY		11	583	10	(20)	00:00:01
* 3	HASH JOIN		102	5406	8	(0)	00:00:01
4	NESTED LOOPS		31	806	5	(0)	00:00:01
5	NESTED LOOPS		31	806	5	(0)	00:00:01
6	VIEW	index\$_join\$_004	3	33	2	(0)	00:00:01
* 7	HASH JOIN						
8	INDEX FAST FULL SCAN	PK_PLAN_SUSCRIPCION	3	33	1	(0)	00:00:01
9	INDEX FAST FULL SCAN	UQ_PLAN_NOMBRE	3	33	1	(0)	00:00:01
* 10	INDEX RANGE SCAN	IX_USUARIO_PLAN	10		0	(0)	00:00:01
11	TABLE ACCESS BY INDEX ROWID	USUARIO	10	150	1	(0)	00:00:01
* 12	TABLE ACCESS FULL	PAGO_SUSCRIPCION	102	2754	3	(0)	00:00:01

Predicate Information (identified by operation id):

```

3 - access("U"."ID_USUARIO"="PG"."ID_USUARIO")
7 - access(ROWID=ROWID)
10 - access("PL"."ID_PLAN"="U"."ID_PLAN")
12 - filter("PG"."FECHA_PAGO"<=TIMESTAMP' 2025-12-31 23:59:59.000000000' AND
          "PG"."ESTADO_PAGO"='EXITOSO' AND "PG"."FECHA_PAGO">=TIMESTAMP' 2025-01-01
          00:00:00.000000000')
```

Note

- this is an adaptive plan

Consulta 4 Reproducciones por tipo de contenido

Cuenta cuántas reproducciones totales tiene cada tipo de contenido (ej. película, serie, documental), ordenando de mayor a menor para identificar los más consumidos.

```
-- 4) Conteo de reproducciones por tipo de contenido
DELETE FROM plan_table;

EXPLAIN PLAN FOR
SELECT
    c.tipo_contenido,
    COUNT(*) AS total_reproducciones
FROM   reproduccion r
JOIN   contenido   c ON c.id_contenido = r.id_contenido
GROUP BY c.tipo_contenido
ORDER BY total_reproducciones DESC;

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
/
```

Antes

```
PLAN_TABLE_OUTPUT
-----
Plan hash value: 689703506

-----
| Id | Operation                | Name           | Rows  | Bytes | Cost (%CPU)| Time     |
-----
|  0 | SELECT STATEMENT         |                |      5 |    90 |       7 (15)| 00:00:01 |
|  1 |   SORT ORDER BY          |                |      5 |    90 |       7 (15)| 00:00:01 |
|  2 |    HASH GROUP BY         |                |      5 |    90 |       7 (15)| 00:00:01 |
|*  3 |     HASH JOIN             |                |    304 |   5472 |        6 (0)| 00:00:01 |
|  4 |      VIEW                 | VW_GBF_7       |     40 |    600 |        3 (0)| 00:00:01 |
|  5 |       TABLE ACCESS FULL | CONTENIDO      |     40 |    480 |        3 (0)| 00:00:01 |
|  6 |       TABLE ACCESS FULL | REPRODUCCION   |    304 |    912 |        3 (0)| 00:00:01 |
-----

Predicate Information (identified by operation id):
-----

   3 - access("ITEM_1"="R"."ID_CONTENIDO")
```

Después

```
PLAN_TABLE_OUTPUT
-----
Plan hash value: 2796423201

-----
| Id | Operation                | Name           | Rows  | Bytes | Cost (%CPU)| Time     |
-----
|  0 | SELECT STATEMENT         |                |      5 |    75 |       3 (34)| 00:00:01 |
|  1 |   SORT ORDER BY          |                |      5 |    75 |       3 (34)| 00:00:01 |
|  2 |    SORT GROUP BY NOSORT  |                |      5 |    75 |       3 (34)| 00:00:01 |
|  3 |     NESTED LOOPS         |                |    304 |   4560 |        2 (0)| 00:00:01 |
|  4 |      TABLE ACCESS BY INDEX ROWID | CONTENIDO      |     40 |    480 |        2 (0)| 00:00:01 |
|  5 |       INDEX FULL SCAN    | IX_CONTENIDO_TIPO |     40 |        |        1 (0)| 00:00:01 |
|*  6 |       INDEX RANGE SCAN   | IX_REPRODUCCION_CONTENIDO |      8 |    24 |        0 (0)| 00:00:01 |
-----

Predicate Information (identified by operation id):
-----

   6 - access("C"."ID_CONTENIDO"="R"."ID_CONTENIDO")
```

Resultados

Consulta 1 Historial de reproducciones por perfil y fecha

Aspecto	Sin índice	Con índice
Operación	TABLE ACCESS FULL	INDEX RANGE SCAN + TABLE ACCESS BY ROWID
Índice usado	—	IX_REPRODUCCION_PERFIL
Costo	4	3
Filas estimadas	2	2

Mejora: El optimizador deja de recorrer toda la tabla y usa el índice para localizar directamente las filas del perfil 1, reduciendo el costo un 25%.

Consulta 2 Usuarios por ciudad

Aspecto	Sin índice	Con índice
Operación	TABLE ACCESS FULL + HASH GROUP BY	INDEX FULL SCAN + SORT GROUP BY NOSORT
Índice usado	—	IX_USUARIO_CIUDAD
Costo	5	2
Filas estimadas	31	31

Mejora: Al estar la ciudad ya ordenada en el índice, Oracle evita leer la tabla y elimina la necesidad de ordenar para agrupar (**NOSORT**), reduciendo el costo un 60%.

Consulta 3 Ingresos por ciudad y plan

Aspecto	Sin índice	Con índice
Operación	HASH JOIN + HASH JOIN	NESTED LOOPS + INDEX RANGE SCAN
Índice usado	—	IX_USUARIO_PLAN
Costo	10	10
Filas estimadas	102	102

Mejora marginal: El costo total se mantiene en 10, pero el join entre PLAN_SUSCRIPCION y USUARIO pasa de HASH JOIN a NESTED LOOPS con IX_USUARIO_PLAN, lo que es más eficiente para tablas pequeñas. PAGO_SUSCRIPCION sigue con FULL SCAN porque el volumen filtrado sigue siendo alto.

Consulta 4 Reproducciones por tipo de contenido

Aspecto	Sin índice	Con índice
Operación	HASH JOIN + TABLE ACCESS FULL x2	NESTED LOOPS + INDEX FULL SCAN + INDEX RANGE SCAN
Índices usados	—	IX CONTENIDO_TIPO, IX REPRODUCCIÓN CONTENIDO
Costo	7	3
Filas estimadas	304	304

Mejora: Oracle reemplazados full scans y un hash join por un recorrido de índices con NESTED LOOPS, bajando el costo un 57%. Además usa SORT GROUP BY NOSORT gracias al orden del índice.

Usuarios y Roles

Se definieron cuatro roles y usuarios siguiendo el principio de mínimo privilegio, es decir, cada usuario del sistema accede únicamente a los datos y operaciones que su función laboral requiere. Esto reduce el riesgo de modificaciones accidentales o malintencionadas, y refleja la separación natural de responsabilidades dentro de una plataforma de streaming real.

Roles

rol_admin: Acceso completo de lectura y escritura sobre todas las tablas del esquema, además de ejecución de todos los procedimientos y funciones. Pensado para administración operacional del sistema.

rol_analista: Sólo lectura sobre todas las tablas y vistas materializadas. Diseñado para consultar datos y generar reportes sin riesgo de alterarlos.

rol_soporte: Acceso limitado a las tablas relacionadas con usuarios, pagos y reproducciones. Puede consultar y gestionar pagos, y ejecutar el cambio de plan. Orientado a la atención al cliente.

rol_contenido: Gestión completa del catálogo: contenido, géneros, temporadas y episodios. Solo lectura sobre reproducciones y calificaciones para monitorear el consumo sin modificarlo.

Usuarios

usr_admin: asignado a rol_admin, representa al administrador del sistema, con capacidad de gestionar cualquier entidad del esquema y ejecutar todos los procedimientos definidos.

usr_analista: asignado a rol_analista, representa al equipo de análisis de datos o business intelligence, con acceso de solo lectura para construir reportes e indicadores sin afectar la operación.

usr_soporte: asignado a rol_soporte, representa al agente de servicio al cliente, con acceso restringido a la información necesaria para resolver problemas de cuenta, pagos y planes de usuarios.

usr_contenido: asignado a rol_contenido, representa al equipo editorial o de catálogo, encargado de administrar el contenido disponible en la plataforma sin acceso a datos financieros ni de usuarios.

```
CREATE ROLE rol_admin;  
CREATE ROLE rol_analista;  
CREATE ROLE rol_soporte;  
CREATE ROLE rol_contenido;
```

```
CREATE USER usr_admin IDENTIFIED BY Admin2026  
  DEFAULT TABLESPACE tbs_quindioflix  
  QUOTA 100M ON tbs_quindioflix;  
GRANT CREATE SESSION TO usr_admin;  
GRANT rol_admin      TO usr_admin;  
  
CREATE USER usr_analista IDENTIFIED BY Analista2026  
  DEFAULT TABLESPACE tbs_quindioflix  
  QUOTA 50M ON tbs_quindioflix;  
GRANT CREATE SESSION TO usr_analista;  
GRANT rol_analista    TO usr_analista;  
  
CREATE USER usr_soporte IDENTIFIED BY Soporte2026  
  DEFAULT TABLESPACE tbs_quindioflix  
  QUOTA 50M ON tbs_quindioflix;  
GRANT CREATE SESSION TO usr_soporte;  
GRANT rol_soporte     TO usr_soporte;  
  
CREATE USER usr_contenido IDENTIFIED BY Contenido2026  
  DEFAULT TABLESPACE tbs_quindioflix  
  QUOTA 50M ON tbs_quindioflix;  
GRANT CREATE SESSION TO usr_contenido;  
GRANT rol_contenido  TO usr_contenido;
```

El script sql completo esta adjunto a los archivos de esta entrega como: UsuariosRoles.sql

Pruebas de restricciones

Usuario analista

Lo que debe funcionar: contar el total de registros en pago_suscripcion

```

1 Show user
2 -- 1) Debería funcionar:
3 SELECT COUNT(*) FROM quindioflix.pago_suscripcion;

```

Salida de Script x

Tarea terminada en 0,035 segundos

USER es "USR_ANALISTA"

COUNT (*)
157

Lo que NO debe funcionar: insertar un nuevo usuario en la tabla usuario, porque su rol solo permite lectura, no escritura

```

1 Show user
2 -- 2) Debería FALLAR (sin privilegios DML):
3 INSERT INTO quindioflix.usuario (id_plan, nombre, apellido, email,
4 fecha_nacimiento, ciudad_residencia)
5 VALUES (1, 'Prueba', 'Analista', 'prueba.analista@uqvirtual.edu.co',
6 DATE '2000-01-01', 'Armenia');

```

Salida de Script x

Tarea terminada en 0,071 segundos

USER es "USR_ANALISTA"

Error que empieza en la línea: 3 del comando -

```

INSERT INTO quindioflix.usuario (id_plan, nombre, apellido, email,
    fecha_nacimiento, ciudad_residencia)
VALUES (1, 'Prueba', 'Analista', 'prueba.analista@uqvirtual.edu.co',
    DATE '2000-01-01', 'Armenia')

```

Error en la línea de comandos : 3 Columna : 29

Informe de error -

Error SQL: ORA-01031: privilegios insuficientes

[https://docs.oracle.com/error-help/db/ora-01031/01031.00000 - "insufficient privileges"](https://docs.oracle.com/error-help/db/ora-01031/01031.00000 - \)

*Document: YES

*Cause: A database operation was attempted without the required privilege(s).

*Action: Ask your database administrator or security administrator to grant you the required privilege(s).

More Details :

<https://docs.oracle.com/error-help/db/ora-01031/>

Para usuario soporte

Lo que debe funcionar: registrar un nuevo pago en pago_suscripcion

```

1  show user
2  -- 1) Debería funcionar:
3  INSERT INTO quindioflix.pago_suscripcion (
4      id_usuario, fecha_pago, periodo_inicio, periodo_fin,
5      monto_base, porcentaje_descuento, monto_pagado,
6      metodo_pago, estado_pago
7  ) VALUES (
8      1, SYSTIMESTAMP, TRUNC(SYSDATE), TRUNC(SYSDATE)+30,
9      24900, 0, 24900,
10     'PSE', 'EXITOSO'
11 );

```

Salida de Script x

Tarea terminada en 0,092 segundos

USER es "USR_SOPORTE"

1 fila insertadas.

Lo que no debe funcionar: eliminar un usuario de la tabla usuario porque no tiene DELETE sobre usuarios ya que esa operación está reservada al administrador

```

1  show user
2  -- 2) Debería FALLAR:
3  DELETE FROM quindioflix.usuario WHERE id_usuario = 1;
4

```

Salida de Script x

Tarea terminada en 0,046 segundos

USER es "USR_SOPORTE"

Error que empieza en la línea: 3 del comando -
DELETE FROM quindioflix.usuario WHERE id_usuario = 1
Error en la línea de comandos : 3 Columna : 29
Informe de error -
Error SQL: ORA-01031: privilegios insuficientes

<https://docs.oracle.com/error-help/db/ora-01031/01031.00000> - "insufficient privileges"

*Document: YES

*Cause: A database operation was attempted without the required privilege(s).

*Action: Ask your database administrator or security administrator to grant you the required privilege(s).

More Details :
<https://docs.oracle.com/error-help/db/ora-01031/>

Para usuario contenido

Lo que debe funcionar: insertar una nueva película en la tabla contenido

```

1 show user
2 -- 1) Deberia funcionar:
3 INSERT INTO quindioflix.contenido (
4     id_empleado_publica,
5     tipo_contenido,
6     titulo,
7     anio_lanzamiento,
8     clasificacion_edad,
9     fecha_agregado,
10    es_original,
11    activo
12 ) VALUES (
13     5,
14     'PELICULA',
15     'Prueba NTS',
16     2025,
17     '+13',
18     SYSDATE,
19     'N',
20     'S'
21 );

```

Salida de Script x

Tarea terminada en 0,078 segundos

USER es "USR_CONTENIDO"

1 fila insertadas.

Lo que no debe funcionar: modificar el monto de un pago en pago_suscripcion porque no tiene ningún privilegio sobre tablas financieras.

```

1 show user
2 -- 2) Deberia FALLAR:
3 UPDATE quindioflix.pago_suscripcion
4 SET monto_pagado = 0
5 WHERE id_pago = 1;

```

Salida de Script x

Tarea terminada en 0,093 segundos

USER es "USR_CONTENIDO"

Error que empieza en la linea: 3 del comando -

```

UPDATE quindioflix.pago_suscripcion
SET monto_pagado = 0
WHERE id_pago = 1

```

Error en la linea de comandos : 3 Columna : 24

Informe de error -

Error SQL: ORA-00942: la tabla o vista no existe

[https://docs.oracle.com/error-help/db/ora-00942/00942.00000 - "table or view does not exist"](https://docs.oracle.com/error-help/db/ora-00942/00942.00000 -)

*Cause: The specified table or view did not exist, or a synonym pointed to a table or view that did not exist. To find existing user tables and views, query the ALL_TABLES and ALL_VIEWS data dictionary views. Certain privileges may be required to access the table. If an application returned this message, then the table that the application tried to access did not exist in the database, or the application did not have access to it.

*Action: Check each of the following

- The spelling of the table or view name is correct.
- The referenced table or view name does exist.
- The synonym points to an existing table or view.

More Details :

<https://docs.oracle.com/error-help/db/ora-00942/>